

PostgreSQL Benchmark Report

TPC-C (OLTP) · TPC-H SF10 & SF100 (OLAP)

Generated June 09, 2026 · github.com/skariapaul/postgresql-bench

973,227 NOPM TPC-C New Orders / min	19,316 QphH@SF100 TPC-H composite score	15,368 Power@SF100 Geo mean 23.4s / query	24,280 Throughput@SF100 5 streams · 1631s
--	--	--	--

Reference System

Component	Detail
CPU	AMD EPYC 9555P — 64 cores / 128 threads, single socket
NUMA	NPS4 — 4 nodes; benchmarks pinned to node 0 (CPUs 0-15, 64-79, 32 logical CPUs)
RAM	377 GiB DDR5
Storage	465 GB NVMe SSD
OS	Ubuntu 24.04 LTS
PostgreSQL	17 (Docker image postgres:17)
HammerDB	5.0

TPC-C — OLTP Benchmark

TPC-C simulates an order-entry workload with five transaction types (New Order, Payment, Order Status, Delivery, Stock Level). The primary metric is **NOPM** — New Orders Per Minute.

Parameter	Value	Parameter	Value
Warehouses	64	shared_buffers	80 GB
Build VUs	8	synchronous_commit	off
Run VUs	32	jit	off
Ramp-up	2 min	max_parallel_workers	64
Timed run	10 min	effective_cache_size	280 GB

VUs	Warehouses	NOPM	TPM	Config
32	64	973,227	2,240,247	sync_commit=off, jit=off, shared_buffers=80GB

973,227 NOPM is the sustained New-Order transaction rate over the 10-minute timed window after a 2-minute ramp-up.

TPC-H — OLAP Benchmark

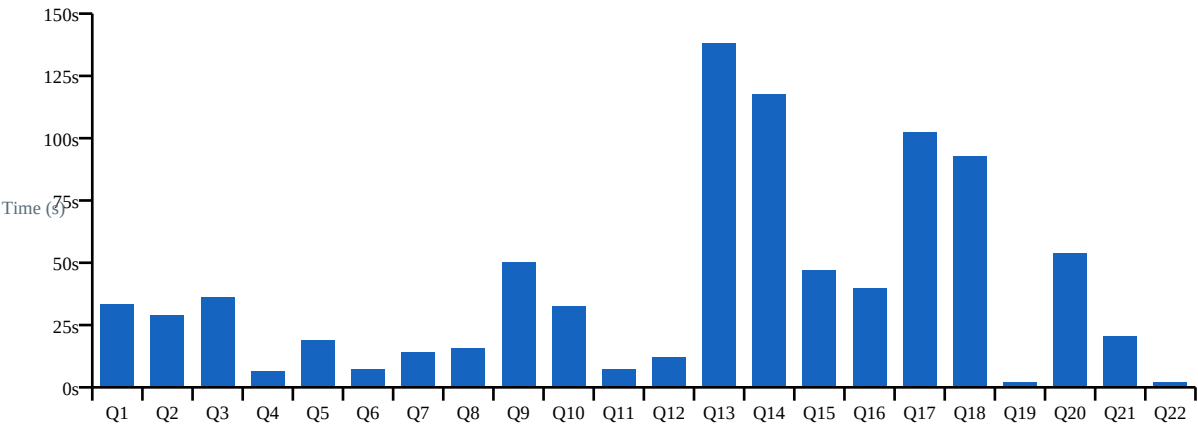
TPC-H measures decision-support performance using 22 complex analytical queries over a synthetic supply-chain database. The composite metric is **QphH@Size** = $\sqrt{(\text{Power@Size} \times \text{Throughput@Size})}$. Higher is better.

Parameter	Value	Parameter	Value
shared_buffers	100 GB	wal_level	minimal
work_mem	4 GB	autovacuum	off
jit	on	max_parallel_workers	64
Power test	1 VU, dop=8	Throughput test	5 VUs, dop=3

Scale	Test	Total Time	Geo Mean	Score
SF10	Power	78 s	1.88 s	—
SF10	Throughput	~105 s	2.75 s	—
SF100	Power	879 s	23.4 s	15,375
	Throughput	1,631 s	42.9 s	24,280

QphH@SF100 = $\sqrt{(15,368 \times 24,280)}$ = 19,316

SF100 Power Test — Per-Query Execution Times



Query execution times for the SF100 power test (1 VU, dop=8). Q13 (138 s) and Q14 (118 s) are the longest — both involve large aggregations over the 600M-row lineitem table.

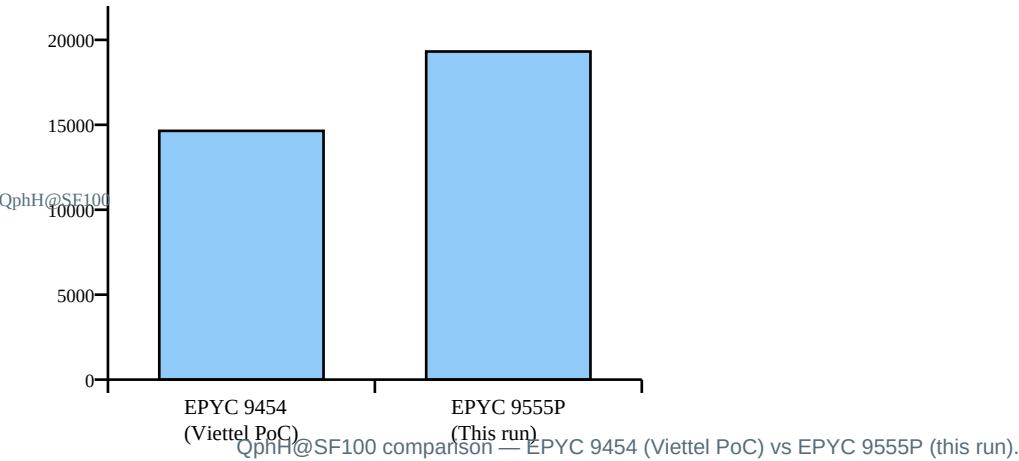
Query	Time (s)	Query	Time (s)	Query	Time (s)
Q1	33.2	Q2	28.8	Q3	36.1
Q4	6.5	Q5	18.8	Q6	7.4
Q7	14.0	Q8	15.7	Q9	50.1
Q10	32.7	Q11	7.1	Q12	12.0
Q13	138.2	Q14	117.8	Q15	47.2
Q16	39.9	Q17	102.6	Q18	92.6

Q19	2.1	Q20	53.9	Q21	20.5
Q22	2.1				

Comparison with Viettel PoC Reference

The Viettel PoC document benchmarked an AMD EPYC 9454 system at SF100 and reported QphH@SF100 = 14,644. Our EPYC 9555P result of **19,316** is **32% higher**, consistent with the generational improvement from the Genoa (9004) to Genoa-X/next-gen architecture and the higher core frequency of the 9555P (3.1 GHz base vs 2.25 GHz for 9454).

System	CPU	Cores (bench)	QphH@SF100	vs Viettel
Viettel PoC	EPYC 9454	—	14,644	baseline
This run	EPYC 9555P	32 (NPS4 node0)	19,316	+32%



Key Tuning Decisions

Setting	Value	Rationale
<code>synchronous_commit = off</code>	TPC-C only	Allows WAL writer to batch flushes; dramatically improves OLTP throughput. Data is not lost on PostgreSQL crash (WAL is still written).
<code>jit = off</code>	TPC-C only	JIT compilation overhead exceeds savings for short OLTP transactions. Enabled for TPC-H where queries are long-running.
<code>shared_buffers = 100 GB</code>	TPC-H	Keeps the hot portion of the 103 GB lineitem heap in memory after warm-up, avoiding repeated disk I/O during analytical queries.
<code>wal_level = minimal</code>	TPC-H build	Eliminates WAL overhead for load-only workloads; CREATE INDEX uses the fast non-WAL path. Replication disabled (<code>max_wal_senders=0</code>).
<code>--cpuset-cpus</code> only (no <code>--cpuset-mems</code>)	Both	CPU pinning to NUMA node 0 maintains cache locality without restricting memory to node 0's ~94 GB. With <code>--cpuset-mems 0</code> , the kernel OOM-kills PostgreSQL when <code>shared_buffers</code> exceeds available node memory during checkpoints.
<code>dop = 8 (power)</code> / <code>dop = 3 (throughput)</code>	TPC-H	Power test uses higher parallelism since only one stream runs. Throughput test uses lower dop so 5 concurrent streams share CPU resources.

Reproducing These Results

All scripts, configurations, and this report are available at github.com/skariapaul/postgresql-bench. The runner scripts auto-detect RAM, CPU count, and NUMA topology and scale all parameters accordingly — no manual tuning required.

Path	Purpose
benchmarks/postgresql/run_tpcc.sh	Auto-scaling TPC-C runner (detects RAM/CPU/NUMA)
benchmarks/postgresql/run_tpch.sh	Auto-scaling TPC-H runner — arg: sf10 sf100
benchmarks/postgresql/docker/tpcc/	OLTP-tuned postgresql.conf (reference)
benchmarks/postgresql/docker/tpch/	OLAP-tuned postgresql.conf (reference)
benchmarks/postgresql/tcl/	HammerDB TCL scripts for build / power / throughput
benchmarks/postgresql/results/	Run logs (gitignored) and this report

Quick-start:

```
git clone https://github.com/skariapaul/postgresql-bench && cd
postgresql-bench/benchmarks/postgresql
HAMMERDB_DIR=/path/to/HammerDB ./run_tpcc.sh
HAMMERDB_DIR=/path/to/HammerDB ./run_tpch.sh sf100
```

This is an informal benchmark for hardware evaluation purposes. Results are not TPC-audited. All configurations, logs, and scripts are publicly available. Report generated 2026-06-09 15:16 UTC.